Tratamiento de datos.

Caso práctico



Ana y **Juan** continúan trabajando en la aplicación web de juegos online. Ahora deben empezar a gestionar la información que generará la propia aplicación.

—Necesitaremos registrar nuevos usuarios, juegos y partidas, pero también modificar sus datos o eliminarlos cuando sea necesario —comenta **Juan**.

—Y tendremos que asegurarnos de que esos cambios no provoquen datos incorrectos —responde **Ana**—. Por ejemplo, una partida no debería estar asociada a un juego que no existe.

—Además, al ser una aplicación online, varios usuarios podrán realizar operaciones al mismo tiempo —añade **Juan**—. Tendremos que controlar qué ocurre si dos procesos intentan modificar los mismos datos.

A lo largo del desarrollo comprobarán que tratar los datos no consiste únicamente en **insertarlos, modificarlos o eliminarlos**, sino también en asegurar que cada operación se realice de forma **coherente**, pueda **confirmarse o deshacerse** cuando sea necesario y funcione correctamente en un entorno **multiusuario**.

Una base de datos debe permitir que la información almacenada pueda **cambiar a lo largo del tiempo**. Para ello, los datos pueden editarse mediante **herramientas gráficas** o mediante **sentencias SQL**, según las necesidades de cada operación.

Sin embargo, cualquier modificación debe mantener la **coherencia de la información** y respetar las relaciones existentes entre las tablas. También es necesario disponer de mecanismos que permitan **confirmar o deshacer cambios** cuando una operación no finalice correctamente.



Magnific. *Tratamiento de fallos.* (CC BY-NC-SA)

En aplicaciones utilizadas por varios usuarios, además, pueden ejecutarse distintas operaciones de forma simultánea. El SGBD debe gestionar esta **concurrencia** para evitar que unas transacciones interfieran incorrectamente con otras.

En esta unidad se trabajará con los mecanismos necesarios para realizar estas operaciones de forma **segura**, **controlada** y **consistente**.

? Preguntas para comenzar la unidad

1. ¿Qué **operaciones** crees que debe realizar una aplicación para mantener actualizada la **información almacenada** en una base de datos?

2. ¿Qué podría ocurrir si se elimina un **registro** que todavía está relacionado con datos de **otra tabla**?
3. ¿Por qué puede ser necesario **deshacer varias modificaciones** cuando solo una de las operaciones de un proceso ha fallado?
4. ¿Qué problemas podrían aparecer si **dos usuarios** intentan modificar el **mismo dato** al mismo tiempo?



Materiales formativos de FP Online propiedad del Ministerio de Educación, Formación Profesional y Deportes.

Aviso Legal

1.- Introducción.

Caso práctico



Juan quiere que **Ana**, la alumna en prácticas, empiece a trabajar con los datos almacenados en la base de datos de la empresa. Para ello, le pregunta qué mecanismos conoce para poder **añadir**, **modificar** o **eliminar** información. Ana recuerda que estas operaciones pueden realizarse mediante **sentencias SQL**, que ofrecen un mayor control, pero también mediante las herramientas visuales del sistema gestor, que resultan más cómodas para determinadas tareas. Sin embargo, sabe que en ambos casos será necesario mantener la **coherencia** de los datos y respetar las **relaciones** existentes entre las tablas.

El **tratamiento de datos** comprende las operaciones de **inserción**, **modificación** y **eliminación** de información en una base de datos, garantizando su **integridad**, su **consistencia** y su correcto funcionamiento en entornos **multiusuario**.

Las bases de datos no solo permiten **almacenar** información, sino también realizar operaciones sobre ella para mantenerla actualizada y útil. Entre las acciones más habituales se encuentran **insertar** nuevos registros, **modificar** datos ya existentes y **eliminar** aquellos que ya no deban conservarse.



Victor C.. *Tratamiento de datos.* (CC BY-NC-SA)

En esta unidad se estudiarán los principales mecanismos para realizar ese tratamiento de los datos. Por un lado, existen **herramientas gráficas** que facilitan estas operaciones de forma más visual. Por otro, pueden utilizarse **sentencias SQL**, que permiten trabajar de una forma más precisa, flexible y detallada.

Estas operaciones no deben ejecutarse de forma arbitraria. Al tratar los datos, es necesario respetar la **estructura** de la base de datos y las **relaciones** definidas entre sus tablas. De este modo, se evita introducir valores incorrectos o provocar situaciones que incumplan la **integridad referencial**.

Además, cuando una operación falla o un usuario decide cancelar un cambio, la base de datos debe poder mantenerse en un estado correcto. Para ello resultan fundamentales las **transacciones**, que permiten confirmar o deshacer operaciones, y los mecanismos de **concurrencia**, que ayudan a controlar el acceso simultáneo a la información.

Por tanto, el tratamiento de datos implica actuar sobre la información almacenada de forma **segura**, **coherente** y **fiable**, especialmente cuando varios usuarios o aplicaciones trabajan al mismo tiempo sobre la misma base de datos.

Autoevaluación

Una aplicación de juegos online permite registrar usuarios, modificar su crédito y eliminar partidas. Además, varios usuarios pueden utilizarla simultáneamente y algunas operaciones pueden cancelarse antes de finalizar.

¿Qué mecanismos serán necesarios para gestionar correctamente estas operaciones?

- [ ] a. utilizar mecanismos que permitan insertar, modificar y eliminar los datos almacenados.

- [ ] b. impedir cualquier modificación cuando haya más de un usuario conectado.

- [ ] c. mantener las relaciones entre los datos para evitar referencias a registros inexistentes.

- [ ] d. disponer de mecanismos que permitan confirmar o deshacer determinadas operaciones.

Mostrar retroalimentación

Solución

1. Correcto
2. Incorrecto
3. Correcto
4. Correcto

2.- Edición de la información mediante herramientas gráficas.

Caso práctico

Ana ha recibido el encargo de introducir algunos datos en las tablas de la base de datos para realizar distintas pruebas de funcionamiento. Como se trata de pocos registros, decide utilizar una **herramienta gráfica**, que le permitirá consultar las tablas e introducir o modificar los datos de una forma rápida y visual.



Ana sabe que también podría realizar estas operaciones mediante **sentencias SQL**, pero para esta tarea concreta considera más cómodo trabajar directamente sobre los datos desde la interfaz del sistema gestor.

Las **herramientas gráficas** permiten **consultar**, **insertar**, **modificar** y **eliminar** datos mediante una **interfaz visual**, facilitando especialmente las operaciones sencillas o realizadas sobre pocos registros.

Los SGBD pueden proporcionar **herramientas gráficas** para trabajar directamente con los datos de las tablas. Estas resultan especialmente útiles para realizar operaciones **manuales y puntuales** sobre un número reducido de registros.

Desde estas herramientas se pueden realizar operaciones como:

- **Insertar** nuevos registros.
- **Modificar** los valores de registros existentes.
- **Eliminar** registros.
- **Consultar** los datos almacenados.
- **Confirmar** o deshacer cambios pendientes.



Magnific. *Edición de la información mediante herramientas gráficas.* (CC BY-NC-SA)

Aunque la operación se realice desde una interfaz gráfica, los datos deben seguir cumpliendo las **restricciones** definidas en las tablas. Por ejemplo, no será posible introducir un valor incompatible con el tipo de dato o eliminar un registro cuando se incumpla una restricción de **integridad referencial**.

En Oracle puede utilizarse **SQL Developer** para visualizar tablas, editar sus datos y ejecutar **sentencias SQL**. Para operaciones repetitivas, complejas o que deban incorporarse posteriormente a una aplicación, resulta más adecuado trabajar directamente con SQL.

Importante: Las herramientas gráficas facilitan la edición manual de los datos, pero **no sustituyen el conocimiento de SQL**.



Para saber más

Para obtener más información sobre SQLDeveloper puedes acceder a la web oficial. Agrégalo a marcadores para tenerlo a mano.

Acceso a manual en la web oficial de SQLDeveloper



Reflexiona

Ana tiene que introducir **seis registros de prueba** y corregir posteriormente dos valores. Juan le propone hacerlo desde SQL Developer, pero también podrían preparar varias sentencias SQL.

¿Qué opción sería más adecuada en este caso y cuándo tendría más sentido utilizar directamente sentencias SQL?

Mostrar retroalimentación

Para una operación **puntual** y sobre pocos registros, utilizar la interfaz gráfica de **SQL Developer** puede ser más rápido y cómodo.

Las **sentencias SQL** serían más adecuadas cuando sea necesario trabajar con muchos registros, repetir las operaciones, conservarlas en un script o incorporarlas posteriormente al código de una aplicación.

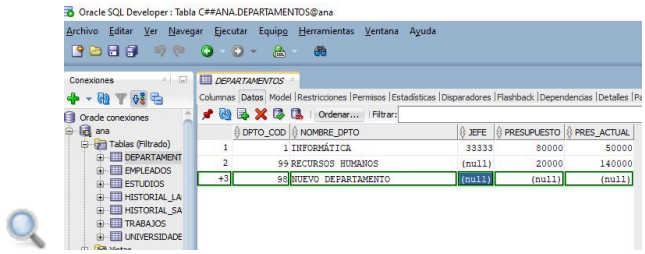
La elección depende principalmente del **volumen**, la **complejidad** y la necesidad de **reutilizar** las operaciones.

2.1.- Inserción de registros.

La **inserción de registros** permite añadir nuevas **filas** a una tabla, respetando los **tipos de datos**, las **restricciones** y las reglas definidas en su estructura.

Desde la interfaz gráfica de **SQL Developer** es posible introducir nuevos registros directamente en una tabla. Para ello, se selecciona la tabla, se accede a la pestaña **Datos** y se utiliza la opción para **insertar una nueva fila**.  .

Una vez creada la fila, pueden introducirse los valores correspondientes en cada una de sus columnas.



Inserción de registros. (CC BY-NC-SA)

Al insertar los datos, el SGBD comprueba que los valores introducidos sean válidos.

Entre otros aspectos, debe comprobarse:

- **Tipo de dato.** El valor introducido debe ser compatible con el tipo definido para la columna.
- **Restricciones.** Deben respetarse las condiciones establecidas mediante **NOT NULL**, **UNIQUE**, **PRIMARY KEY**, **FOREIGN KEY** u otras restricciones.
- **Integridad referencial.** Los valores de las claves foráneas deben hacer referencia a registros válidos cuando así lo exija la relación.

Si alguna condición no se cumple, la inserción será rechazada y se mostrará un **mensaje de error** que permitirá identificar el problema.

Una vez introducidos correctamente los registros, es necesario **confirmar** la operación para que los cambios sean permanentes. Mientras los cambios continúen pendientes, podrán deshacerse mediante **Rollback**.

Autoevaluación

Ana está introduciendo un nuevo registro desde la pestaña **Datos** de SQL Developer. En una columna definida como numérica escribe por error un texto e intenta confirmar la operación.

¿Qué ocurrirá?

- ○ a. oracle guardará el registro y sustituirá automáticamente el texto por null.
- ○ b. oracle mostrará un error y la inserción no se realizará correctamente.
- ○ c. oracle convertirá automáticamente el texto en un valor numérico.

Incorrecta. Oracle no sustituye automáticamente un valor incompatible por **NULL** . El dato introducido debe respetar el **tipo de dato** definido para la columna.

Correcta. Si el valor introducido no es compatible con el tipo de dato de la columna, Oracle mostrará un **mensaje de error** y el registro no podrá insertarse correctamente.

Incorrecta. Oracle no puede convertir cualquier texto en un número. Si el valor no es válido para una columna numérica, se producirá un **error**.

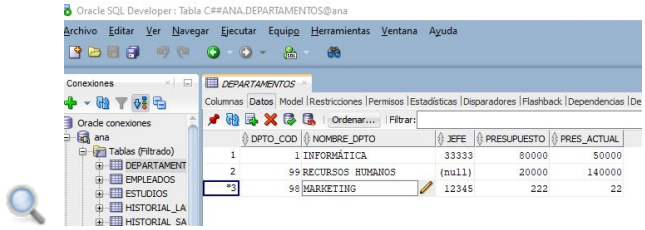
Solución

1. Incorrecto
2. Opción correcta
3. Incorrecto

2.2.- Modificación de registros.

La **modificación de registros** permite cambiar los **valores** almacenados en filas existentes, siempre que los nuevos datos respeten los **tipos de datos** y las **restricciones** de la tabla.

Para modificar un registro desde **SQL Developer**, se selecciona la tabla y se accede a la pestaña **Datos**. A continuación, se localiza la fila correspondiente y se modifica directamente el valor de la columna que se desea actualizar.



Elaboración propia. *Modificación de registros.* (CC BY-NC-SA)

Modificar un valor desde una herramienta gráfica no significa que pueda introducirse cualquier dato. Al aplicar los cambios, el SGBD comprueba que se cumplen las condiciones definidas en la tabla, entre ellas:

- **Tipo de dato.** El nuevo valor debe ser compatible con el tipo definido para la columna.
- **Restricciones.** El cambio debe respetar las restricciones establecidas sobre los datos.
- **Integridad referencial.** Si la columna participa en una relación entre tablas, el nuevo valor debe mantener dicha relación correctamente.

Si alguna de estas condiciones no se cumple, la modificación será **rechazada** y SQL Developer mostrará el error correspondiente. Una vez corregido el problema, podrá intentarse de nuevo la operación.

Cuando las modificaciones sean correctas, deberán **confirmarse** mediante la opción correspondiente para que los nuevos valores queden guardados.

Autoevaluación

Indica si las siguientes situaciones son verdaderas o falsas.

1. Si se modifica desde SQL Developer una columna numérica introduciendo un valor incompatible, el SGBD puede rechazar el cambio aunque se haya realizado desde una interfaz gráfica.

○ Verdadero ○ Falso

Falso

Las herramientas gráficas no eliminan las comprobaciones de la base de datos. El nuevo valor debe respetar el **tipo de dato** y las **restricciones** de la columna.

2. Cambiar de una fila a otra después de editar un valor hace que la modificación quede confirmada automáticamente.

○ Verdadero ○ Falso

Falso

El valor puede permanecer modificado en la interfaz, pero será necesario **confirmar** la operación para que el cambio quede guardado.

3. Una modificación puede ser rechazada si el nuevo valor incumple una relación definida con otra tabla.

○ Verdadero ○ Falso

Falso

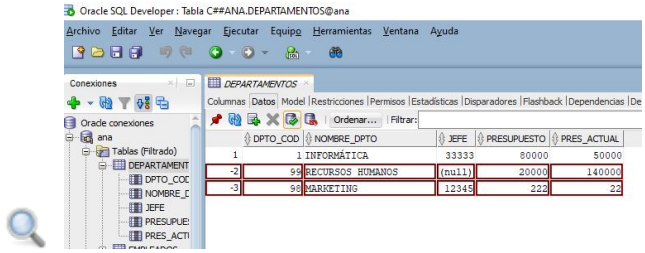
Si el cambio vulnera la **integridad referencial**, el SGBD impedirá que la modificación se aplique.

2.3.- Borrado de registros.

El **borrado de registros** permite eliminar **filas** de una tabla, siempre que la operación respete las **restricciones** y las **relaciones** existentes con otros datos.

Para eliminar un registro desde **SQL Developer**, se selecciona la tabla, se accede a la pestaña **Datos** y se marca la fila que se desea borrar mediante la opción correspondiente.

La fila queda señalada como pendiente de eliminación, pero el borrado no será definitivo hasta que la operación se **confirme**.



Elaboración propia. *Eliminar filas en SQLDeveloper* (CC BY-NC-SA)

Antes de eliminar un registro, el SGBD comprueba que la operación cumple las **restricciones** definidas en la base de datos. Una situación especialmente importante se produce cuando el registro está relacionado con información de otra tabla.

Por ejemplo, si se intenta eliminar un registro que está siendo referenciado desde otra tabla mediante una **clave foránea**, la operación puede ser rechazada para evitar que queden referencias a datos inexistentes.

En Oracle, este tipo de situación puede producir el error **ORA-02292**, que indica que existe un registro relacionado que impide realizar la eliminación.

Cuando el borrado sea válido, será necesario **confirmar** la operación para que la fila desaparezca definitivamente de la tabla.

Autoevaluación

Relaciona cada situación con el resultado correspondiente.

Situación	Respuesta	Opciones
1. Se marca una fila para eliminar, pero todavía no se ha confirmado la operación.		a. el registro queda eliminado definitivamente.
2. Se intenta eliminar un registro al que hace referencia un registro de otra tabla.		b. la fila queda marcada, pero el borrado todavía no es definitivo.

Situación	Respuesta	Opciones
3. Se elimina una fila correctamente y posteriormente se confirma la operación.		c. el borrado puede ser rechazado por la base de datos.

Enviar

1. **b. Correcta.** La fila queda señalada para su eliminación, pero el cambio todavía debe **confirmarse**.
2. **c. Correcta.** Si otro registro hace referencia al que se intenta eliminar, la base de datos puede **rechazar el borrado**.
3. **a. Correcta.** Cuando el borrado es válido y se **confirma**, el registro queda eliminado definitivamente.

3.- Edición de la información mediante sentencias SQL.

Caso práctico



La aplicación web de juegos online que está desarrollando **Juan** necesita modificar directamente la información almacenada en la base de datos. Las herramientas gráficas que ha utilizado **Ana** resultan útiles para operaciones manuales, pero la aplicación deberá realizar estas acciones desde su propio código.

Para ello utilizarán **sentencias SQL** capaces de insertar, modificar y eliminar registros. Antes de incorporarlas a la aplicación, Juan y Ana practicarán su funcionamiento desde **SQL Developer**.

La **edición de la información mediante sentencias SQL** permite **insertar**, **modificar** y **eliminar** registros directamente sobre la base de datos mediante instrucciones del **Lenguaje de Manipulación de Datos (DML)**, como **INSERT**, **UPDATE** y **DELETE**.

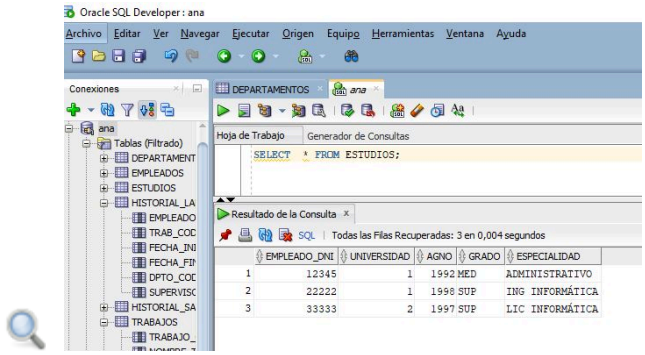
Las operaciones de edición sobre los datos pueden realizarse mediante sentencias del **Lenguaje de Manipulación de Datos (DML)**.

Las principales son:

- **INSERT.** Añade nuevos registros a una tabla.
- **UPDATE.** Modifica los valores de registros existentes.
- **DELETE.** Elimina registros de una tabla.

Estas sentencias permiten realizar las mismas operaciones básicas de edición vistas mediante herramientas gráficas, pero ofrecen mayor control y pueden utilizarse desde **scripts** o desde el código de una aplicación.

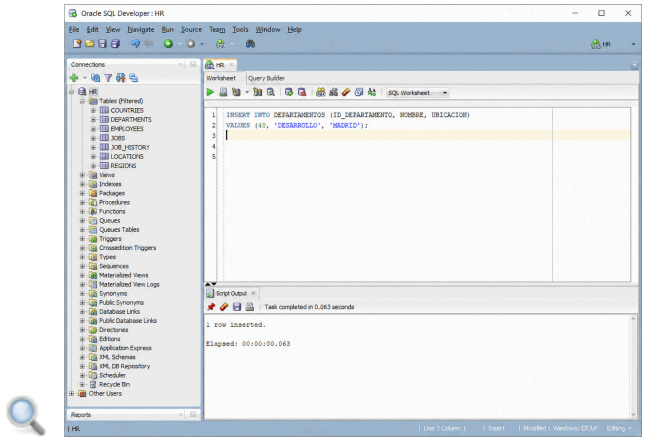
En **SQL Developer**, las sentencias se escriben en una **hoja de trabajo SQL** asociada a una conexión con la base de datos. Desde ella pueden ejecutarse instrucciones y comprobar el resultado o los posibles errores producidos durante su ejecución. La hoja de trabajo continúa siendo el espacio utilizado para escribir y ejecutar sentencias SQL en las herramientas actuales de Oracle.



Elaboración propia. *Hoja de trabajo de SQLDeveloper* (CC BY-NC-SA)

Al ejecutar una sentencia, SQL Developer muestra en la parte inferior el **resultado de la operación** o, si existe algún problema, el correspondiente **mensaje de error**. Esto permite comprobar si la instrucción se ha ejecutado correctamente antes de continuar trabajando.

SQL Developer también mantiene un **historial SQL** desde el que pueden recuperarse sentencias ejecutadas anteriormente. Esta función puede resultar útil para reutilizar o revisar instrucciones ya utilizadas.



Elaboración propia. *Historial SQL.* (CC BY-NC-SA)

Reflexiona

La aplicación de juegos online debe permitir que un usuario se registre, modifique posteriormente algunos de sus datos y elimine una partida que haya creado. Durante las pruebas, **Ana** realiza estas operaciones desde SQL Developer utilizando su interfaz gráfica, pero **Juan** necesita incorporarlas al código de la aplicación.

1. ¿Qué sentencias DML serán necesarias para realizar cada una de estas operaciones?

2. ¿Por qué la aplicación debe utilizar sentencias SQL en lugar de depender de la edición manual desde SQL Developer?

Mostrar retroalimentación

Para registrar un nuevo usuario será necesario utilizar **INSERT**, para modificar sus datos se utilizará **UPDATE** y para eliminar una partida se empleará **DELETE**.

La interfaz gráfica de SQL Developer puede utilizarse para realizar pruebas o modificaciones manuales, pero una aplicación necesita ejecutar estas operaciones de forma **automática desde su código**. Las sentencias SQL permiten que el tratamiento de los datos forme parte del funcionamiento de la propia aplicación sin depender de la intervención manual de un usuario.

3.1.- Inserción de registros.

La **inserción de registros mediante SQL** se realiza con la sentencia **INSERT**, que permite añadir nuevas **filas** a una tabla indicando las **columnas** y los **valores** que se desean almacenar.

La sentencia **INSERT** permite añadir una o varias filas a una tabla.

Su forma básica es:

```
INSERT INTO nombre_tabla (lista_columnas)<br>VALUES (lista_valores);INSERT INTO nombre_tabl
```

En **lista_columnas** se indican las columnas que recibirán datos y, en **lista_valores**, los valores correspondientes. Ambos elementos deben aparecer en el **mismo orden** y separados por comas.

Es recomendable indicar expresamente las columnas que se van a insertar, ya que hace la sentencia más clara y evita depender del orden físico de las columnas de la tabla.

Ejemplo:

```
INSERT INTO USUARIOS (Login, Password, Nombre, Apellidos, Direccion, CP, Localidad, Provinc
F_Ing, Correo, Credito, Sexo) VALUES ('migrod86', '6PX5=V', 'MIGUEL ANGEL', 'RODRIGUEZ RODR
'47001', 'VALLADOLID', 'VALLADOLID', 'ESPAÑA', '27/04/1977', '10/01/2008', 'migrod86@gmail.
```

Cuando se insertan valores de tipo texto, estos deben escribirse entre **comillas simples**. Los valores numéricos se escriben sin comillas.

Para las fechas es recomendable indicar su formato de forma explícita. Por ejemplo, Oracle permite utilizar un literal **DATE**:

```
INSERT INTO USUARIOS(Login, Password, Nombre, Apellidos, F_Nac, F_Ing, Correo)VALUES('migro
'VBROMI', 'NATALIA', 'SANCHEZ GARCIA','C/Blanca','28003','Madrid','Madrid','Spain', 'natsar
```

De esta forma, la inserción no depende del formato de fecha configurado en la sesión.

También pueden omitirse determinadas columnas:

```
INSERT INTO USUARIOS(Login, Password, Nombre, Apellidos, Correo)VALUES('natsan63', 'VBROMI'
```

Cuando una columna no aparece en la sentencia:

- **Valor por defecto.** Se utilizará si la columna tiene definido un valor predeterminado.
- **NULL.** Se utilizará si no existe un valor por defecto y la columna admite **NULL**.
- **Error.** Se producirá si la columna exige un valor y no dispone de un valor por defecto.

La cantidad de columnas indicadas debe coincidir con la cantidad de valores proporcionados. **Por ejemplo:**

```
INSERT INTO USUARIOS (Login, Password, Nombre, Correo)VALUES ('caysan56', 'W4IN5U', 'CAYETA
```

En este caso se han indicado **cuatro columnas** pero **cinco valores**, por lo que Oracle devolverá:

```
ORA-00913: demasiados valores
```



Actividad desplegable

Completa la siguiente sentencia para insertar un producto con código 15, nombre Monitor y precio 189:

INSERT INTO PRODUCTOS (codigo, nombre, precio) [ ] (15, [ ] , 189);

Comprobar | Mostrar retroalimentación

a. **values. Correcta para el primer hueco.** La palabra **VALUES** introduce los valores que se almacenarán en las columnas indicadas.
b. **'Monitor'. Correcta para el segundo hueco.** Al tratarse de un valor de texto, debe escribirse entre **comillas simples**.
c. **Monitor. Incorrecta.** Un texto utilizado como valor debe escribirse entre **comillas simples**; de lo contrario, Oracle no lo interpretará como una cadena de caracteres.
d. **Monitor. Incorrecta.** Al ser un valor de tipo texto, debe escribirse entre **comillas simples** para que Oracle lo interprete correctamente como una cadena de caracteres.

3.2.- Modificación de registros.

La **modificación de registros mediante SQL** se realiza con la sentencia **UPDATE**, que permite cambiar los valores de una o varias **columnas** en todos los registros o únicamente en aquellos que cumplan una **condición**.

La sentencia **UPDATE** permite modificar los valores almacenados en registros existentes de una tabla.

Su sintaxis básica es:

```
UPDATE nombre_tablaSET nombre_columna = valor[WHERE condición];UPDATE nombre_tabla SET nomb
[ WHERE condición ];
```

La cláusula **SET** indica qué columna se modificará y cuál será su nuevo valor. Si se quieren actualizar varias columnas en una misma operación, deben separarse mediante comas:

```
UPDATE USUARIOSSET Credito = 0, F_Nac = NULL;UPDATE USUARIOS SET Credito = 200;
```

En este ejemplo se modifica el **crédito** y la **fecha de nacimiento** de todos los usuarios.

La cláusula **WHERE** permite limitar la modificación a los registros que cumplan una determinada condición.

Ejemplo:

```
UPDATE USUARIOSSET Credito = 300WHERE Sexo = 'M';WHERE Sexo = 'M';UPDATE USUARIOS SET Credi
```

Solo se modificarán los registros que cumplan la condición indicada.

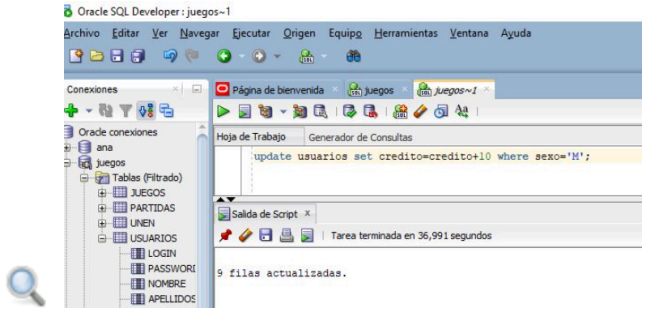
> **Importante:** Si una sentencia **UPDATE** no incluye **WHERE**, la modificación afectará a **todas las filas** de la tabla.

El nuevo valor también puede obtenerse mediante una **expresión** basada en el valor actual de la columna. Por ejemplo, para aumentar en 10 créditos a los usuarios seleccionados:

```
UPDATE USUARIOSSET Credito = Credito + 10WHERE Sexo = 'M';UPDATE USUARIOS SET Credito = 300
```

En este caso no se sustituye el crédito por el valor 10, sino que se **incrementa en 10** el valor que ya tenía cada registro.

Después de ejecutar la sentencia, SQL Developer muestra el **número de filas actualizadas** o el mensaje de error correspondiente si la operación no puede realizarse.



Elaboración propia. *Sentencia update en SQLDeveloper* (CC BY-NC-SA)

Autoevaluación

La tabla **USUARIOS** contiene el campo **Credito**. Se quiere aumentar en **50 créditos** a los usuarios cuyo crédito actual sea inferior a 100, sin modificar al resto.

¿Qué elementos son necesarios para realizar correctamente esta actualización?

- [ ] a. utilizar update usuarios para indicar la tabla que se modificará.
- [ ] b. utilizar set credito = 50 para sustituir el crédito actual por el incremento deseado.
- [ ] c. utilizar set credito = credito + 50 para calcular el nuevo valor a partir del actual.
- [ ] d. utilizar where credito < 100 para limitar los registros afectados.

Mostrar retroalimentación

Solución

1. Correcto

2. Incorrecto
3. Correcto
4. Correcto

3.3.- Borrado de registros.

El **borrado de registros mediante SQL** se realiza con la sentencia **DELETE**, que permite eliminar filas de una tabla y utilizar **WHERE** para determinar qué registros deben borrarse.

La sentencia **DELETE** permite eliminar registros de una tabla.

Su sintaxis básica es:

```
DELETE FROM nombre_tabla[WHERE condición];DELETE FROM nombre_tabla [ WHERE condición ];
```

La cláusula **WHERE** determina qué filas deben eliminarse. Si se omite, **DELETE afectará a todos los registros de la tabla**.

Por ejemplo:

```
DELETE FROM USUARIOS;
```

En este caso solo se eliminarán las filas que cumplan la condición **Credito = 0**.

> **Importante:** Antes de ejecutar una sentencia **DELETE**, es fundamental comprobar la condición **WHERE**, ya que una condición incorrecta o su ausencia puede provocar la eliminación de más registros de los previstos.

El borrado también puede ser rechazado si el registro está siendo referenciado desde otra tabla, tal como se explicó anteriormente al estudiar las **relaciones entre registros**.

Tras ejecutar la sentencia, SQL Developer mostrará el **número de filas eliminadas** o el correspondiente **mensaje de error** si la operación no puede realizarse.



Autoevaluación

Se ejecuta la siguiente sentencia:

```
DELETE FROM USUARIOS;
```

La tabla **USUARIOS** contiene 25 registros.

¿Cuál será el resultado de la operación?

- ○ a. se eliminarán los 25 registros, pero la tabla usuarios seguirá existiendo.

- ○ b. se eliminará completamente la tabla usuarios junto con su estructura.
- ○ c. no se eliminará ningún registro porque falta la cláusula where.

Correcta. Al no existir una cláusula **WHERE**, **DELETE** afecta a todas las filas, pero no elimina la estructura de la tabla.

Incorrecta. DELETE elimina registros, no la tabla. La estructura de **USUARIOS** continúa existiendo.

Incorrecta. La cláusula **WHERE** es opcional. Si no se incluye, la sentencia elimina todos los registros de la tabla.

Solución

1. Opción correcta
2. Incorrecto
3. Incorrecto

4.- Integridad referencial.

Caso práctico



Ana tiene una duda sobre la base de datos de la plataforma de juegos online.

—¿Qué ocurriría si intentáramos crear una partida asociada a un juego que no existe? —pregunta **Ana**.

—La base de datos debe impedirlo mediante la **integridad referencial** —responde **Juan**—. Así podemos asegurar que una partida siempre esté asociada a un juego existente y que su creador corresponda a un usuario registrado.

La **integridad referencial** garantiza que las **relaciones entre tablas** sean válidas, haciendo que los valores de una **clave foránea** correspondan con valores existentes de la **clave referenciada**.

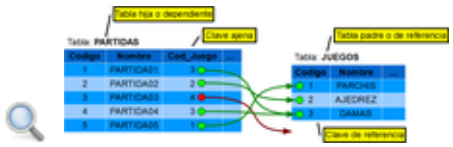
Las tablas de una base de datos relacional pueden relacionarse mediante una **clave foránea**, que contiene valores que hacen referencia a una clave de otra tabla.

En la base de datos de juegos online, la tabla **PARTIDAS** contiene el campo **Cod_Juego**, utilizado para indicar a qué juego pertenece cada partida. Este campo hace referencia al campo **Codigo** de la tabla **JUEGOS**.

Por tanto, cada valor almacenado en **Cod_Juego** debe corresponder con un juego existente.

Ejemplo:

Si una partida contiene **Cod_Juego = 4**, pero en **JUEGOS** no existe ningún registro cuyo **Codigo** sea 4, la relación no será válida y se incumplirá la **integridad referencial**.



Elaboración propia. *Ejemplo Integridad Referencial.* (CC BY-NC-SA)

Para comprender estas relaciones se utilizan los siguientes conceptos:

- **Clave foránea o ajena.** Campo o conjunto de campos de la tabla hija cuyos valores hacen referencia a una clave de otra tabla. En el ejemplo corresponde a **Cod_Juego** de **PARTIDAS**.
- **Clave de referencia.** Clave **primaria** o **única** de la tabla referenciada. En el ejemplo corresponde a **Codigo** de **JUEGOS**.

- **Tabla hija o dependiente.** Tabla que contiene la clave foránea y depende de los valores existentes en la tabla referenciada. En el ejemplo es **PARTIDAS**.
- **Tabla padre o de referencia.** Tabla que contiene la clave a la que hace referencia la clave foránea. En el ejemplo es **JUEGOS**.

De este modo, la integridad referencial evita que aparezcan **referencias a registros inexistentes** y ayuda a mantener la **coherencia de los datos** entre las tablas relacionadas.



Reflexiona

En la tabla **JUEGOS** están registrados únicamente los juegos con códigos **1, 2 y 3**. Ana intenta crear una nueva partida indicando **Cod_Juego = 6**.

1. ¿Debería permitirse guardar esa partida?

2. ¿Qué relación entre las tablas provoca el problema?

3. ¿Qué tendría que ocurrir para que Cod_Juego = 6 fuese un valor válido?

Mostrar retroalimentación

La partida **no debería poder almacenarse** con **Cod_Juego = 6**, ya que ese valor no existe en el campo **Codigo** de la tabla **JUEGOS**.

El problema se produce porque **Cod_Juego** es la **clave foránea** de la tabla hija **PARTIDAS** y debe hacer referencia a un valor existente de la clave de referencia **Codigo** de la tabla padre **JUEGOS**.

Para que **Cod_Juego = 6** fuese válido, tendría que existir previamente en **JUEGOS** un registro cuyo **Codigo** fuese 6. De este modo se mantendría la **integridad referencial**.

4.1.- Integridad en actualización y supresión de registros.

La **integridad en actualización y supresión de registros** controla los cambios realizados sobre **claves foráneas** y **claves referenciadas**, evitando que una modificación o eliminación produzca relaciones inválidas entre las tablas.

La **integridad referencial** también debe mantenerse cuando se modifican o eliminan registros relacionados.

Si se modifica una **clave foránea** de la tabla hija, el nuevo valor debe corresponder con un valor existente de la clave referenciada en la tabla padre, salvo que la columna admita **NULL**.

Ejemplo:

En **PARTIDAS**, el campo **Cod_Juego** hace referencia al campo **Codigo** de **JUEGOS**. Si no existe ningún juego con **Codigo = 4**, no será válido modificar una partida para asignarle **Cod_Juego = 4**.

También pueden producirse problemas cuando se modifica o elimina una fila de la **tabla padre** que está siendo referenciada desde la tabla hija.



Elaboración propia. *Ejemplo Integridad Referencial.* (CC BY-NC-SA)

Ejemplo:

Si varias filas de **PARTIDAS** hacen referencia al juego **DAMAS**, no podrá modificarse su **Codigo** si el cambio deja esas partidas apuntando a un código inexistente.

Del mismo modo, si se intenta eliminar un juego que todavía tiene partidas asociadas, Oracle debe mantener la integridad de la relación.

Para controlar qué ocurre al eliminar una fila de la tabla padre, Oracle permite los siguientes comportamientos:

- **Sin acción de borrado en cascada.** Es el comportamiento predeterminado. Si una fila de la tabla padre está siendo referenciada desde la tabla hija, Oracle impide su eliminación.
- **ON DELETE CASCADE.** Al eliminar una fila de la tabla padre, también se eliminan automáticamente las filas de la tabla hija que la referencian.
- **ON DELETE SET NULL.** Al eliminar una fila de la tabla padre, las claves foráneas correspondientes de la tabla hija pasan a **NULL**. Para utilizar esta opción, la columna debe admitir valores **NULL**.

  Importante: Oracle no proporciona una acción declarativa **ON UPDATE CASCADE** para las claves foráneas. Si se necesita propagar automáticamente un cambio de la clave referenciada, debe resolverse mediante otros mecanismos, como la lógica de la aplicación o disparadores.

Para saber más

Apuntes sobre integridad referencial en Oracle.

Las Restricciones de Integridad en ORACLE.

Actividad desplegable

Relaciona cada situación con el comportamiento adecuado.

Situación	Comportamiento adecuado
1. Al eliminar un juego deben eliminarse también todas las partidas que hacen referencia a él.	
2. Al eliminar un juego, las partidas deben conservarse, pero su **Cod_Juego** debe quedar sin valor.	
3. No debe permitirse eliminar un juego mientras existan partidas que hagan referencia a él.	

Comprobar | Mostrar retroalimentación

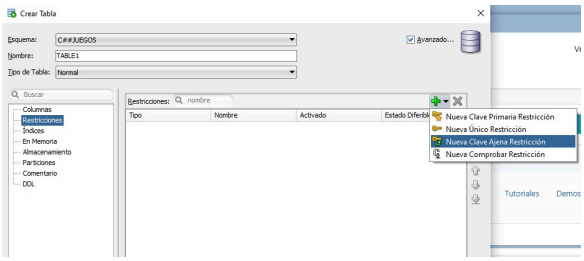
1. **b. ON DELETE CASCADE.** Al eliminar el juego, también se eliminan automáticamente las partidas que hacen referencia a él.
2. **c. ON DELETE SET NULL.** Las partidas se conservan, pero **Cod_Juego** pasa a **NULL**, siempre que la columna permita este valor.
3. **a. Sin acción de borrado en cascada.** Oracle impide eliminar el registro de la tabla padre mientras existan registros de la tabla hija que lo estén referenciando.

4.2.- Supresión en cascada.

La **supresión en cascada** permite configurar una **clave foránea** para que, al eliminar un registro de la **tabla padre**, se eliminen automáticamente los registros relacionados de la **tabla hija** mediante **ON DELETE CASCADE**.

El comportamiento de una **clave foránea** ante la eliminación de un registro de la tabla padre puede definirse durante la creación de la restricción o posteriormente al modificar la estructura de la tabla.

En SQL Developer puede configurarse desde las **restricciones** de la tabla, creando una nueva **clave ajena** y seleccionando la acción que se aplicará cuando se elimine el registro referenciado.



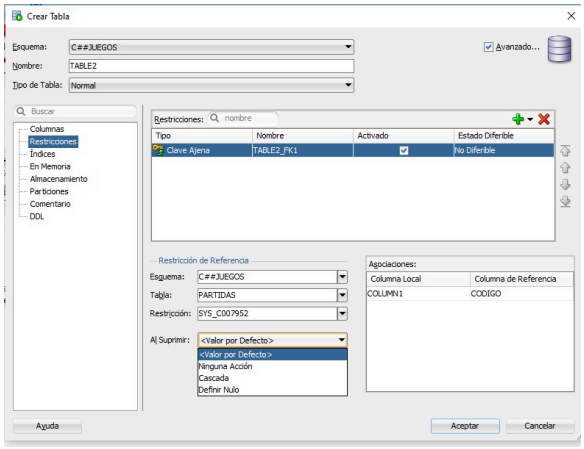
Elaboración propia. *SQLDeveloper elección Nueva clave ajena restricción.* (CC BY-NC-SA)

Ejemplo:

Al crear una clave foránea para relacionar **PARTIDAS** con **JUEGOS**, puede seleccionarse qué ocurrirá cuando se elimine un juego que tenga partidas asociadas.

En el apartado **Al Suprimir** pueden aparecer las siguientes opciones:

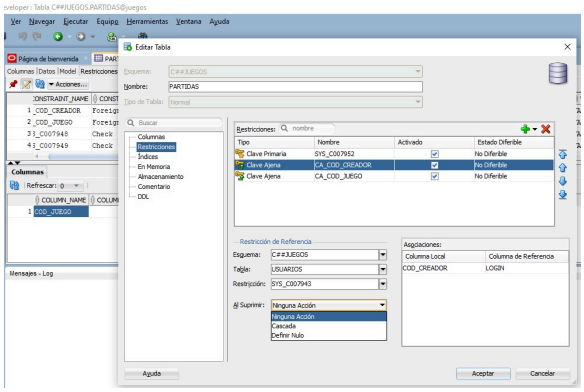
- **Ninguna acción.** Impide eliminar el registro padre mientras existan registros hijos que lo referencien.
- **Cascada.** Elimina también los registros hijos relacionados.
- **Definir nulo.** Mantiene los registros hijos, pero establece su clave foránea a **NULL**, siempre que la columna lo permita.



Elaboración propia. *Opciones al suprimir restricción clave ajena.* (CC BY-NC-SA)

Importante: La opción **<Valor por defecto>** de la interfaz no equivale a una acción **ON DELETE DEFAULT**. Si no se define una acción específica, Oracle utiliza el comportamiento predeterminado de la restricción.

Si la tabla ya existe, la restricción también puede configurarse desde la opción de **editar tabla**, accediendo a sus restricciones y seleccionando la clave foránea correspondiente.



Elaboración propia. *Editar tabla. Restricción clave ajena* (CC BY-NC-SA)

La misma configuración puede realizarse mediante SQL.

Ejemplo:

Para eliminar automáticamente las partidas cuando se elimina el juego al que pertenecen:

```
CONSTRAINT JUEGOS_CONFOREIGN KEY (Cod_Juego)REFERENCES JUEGOS (Codigo)ON DELETE CASCADECONS
```

Hay que recordar que una declaración de este tipo debe hacerse en el momento de crear la tabla (***CREATE TABLE***) o modificar su estructura (***ALTER TABLE***).

Para conservar las partidas y establecer **Cod_Juego** a **NULL**:

```
<span class="cg">CONSTRAINT</span><span> JUEGOS_CON
</span><span class="cg">FOREIGN</span><span> </span><span class="cg">KEY</span><span> (Cod_
</span><span class="cg">REFERENCES</span><span> JUEGOS (Codigo)
</span><span class="cg">ON</span><span> </span><span class="cg">DELETE</span><span> </span>
```

Estas cláusulas pueden incluirse al crear la tabla mediante **CREATE TABLE** o al añadir posteriormente la restricción mediante **ALTER TABLE**. Oracle confirma oficialmente ambas acciones para las claves foráneas



Autoevaluación

La tabla **PARTIDAS** contiene una clave foránea **Cod_Juego** que referencia a **JUEGOS.Codigo**. Se quiere que, cuando se elimine un juego, también desaparezcan automáticamente todas sus partidas.

¿Qué opción debe utilizarse al definir la clave foránea?

- ○ a. on delete set null.
- ○ b. on delete cascade.
- ○ c. ninguna acción.

Incorrecta. ON DELETE SET NULL conservaría las partidas y establecería **Cod_Juego** a **NULL**, siempre que la columna lo permitiera.

Correcta. ON DELETE CASCADE elimina automáticamente los registros de **PARTIDAS** que hacen referencia al juego eliminado.

Incorrecta. Con **ninguna acción**, Oracle impediría eliminar el juego mientras existieran partidas que lo referenciaran.

Solución

1. Incorrecto
2. Opción correcta
3. Incorrecto

5.- Subconsultas y composiciones en órdenes de edición.

Caso práctico



Juan necesita realizar modificaciones en la base de datos utilizando información que se encuentra en otras tablas.

—Quiero aumentar el crédito únicamente a los usuarios que hayan creado alguna partida —comenta **Juan**—. Con una condición sencilla sobre **USUARIOS** no puedo saber quiénes cumplen ese requisito.

—Entonces tendremos que utilizar una **consulta** para localizar primero esos usuarios y emplear su resultado en la operación de actualización —responde **Ana**.

De esta forma podrán realizar operaciones de edición utilizando información obtenida mediante otras consultas.

Las **subconsultas y composiciones en órdenes de edición** permiten utilizar el resultado de una **consulta SELECT** dentro de sentencias **INSERT**, **UPDATE** o **DELETE** para realizar operaciones basadas en información obtenida de otras filas o tablas.

Hasta ahora se han utilizado **INSERT**, **UPDATE** y **DELETE** indicando directamente los valores o las condiciones necesarias para realizar cada operación. Sin embargo, también es posible utilizar una **subconsulta** para obtener esa información.

Una subconsulta es una sentencia **SELECT** incluida dentro de otra sentencia SQL. Su resultado puede utilizarse en las órdenes de edición para realizar operaciones más avanzadas.

Según la operación que se necesite realizar, una subconsulta puede utilizarse para:

- **INSERT.** Obtener mediante una consulta las filas que se insertarán en otra tabla.
- **UPDATE.** Obtener el nuevo valor de una columna o determinar qué registros deben modificarse.
- **DELETE.** Determinar mediante una consulta qué registros deben eliminarse.

Oracle permite estos usos de las subconsultas dentro de las operaciones DML.

Ejemplo:

Supongamos que queremos aumentar el crédito únicamente a los usuarios que aparecen como creadores de alguna partida. Una consulta sobre **PARTIDAS** puede localizar esos usuarios y su resultado puede utilizarse posteriormente para determinar qué registros de **USUARIOS** deben actualizarse.

En los siguientes apartados se estudiará cómo aplicar este mecanismo específicamente a las sentencias **INSERT**, **UPDATE** y **DELETE**.



Reflexiona

Juan quiere aumentar el crédito de los usuarios que hayan creado alguna partida. La información sobre el **crédito** se encuentra en **USUARIOS**, mientras que los creadores de las partidas se encuentran registrados en **PARTIDAS**.

1. ¿Por qué una sentencia UPDATE con una condición basada únicamente en los datos de USUARIOS puede no ser suficiente?

2. ¿Qué utilidad tendría una subconsulta en esta operación?

Mostrar retroalimentación

La condición que determina qué usuarios deben recibir el aumento depende de información almacenada en **PARTIDAS**, por lo que no basta con consultar únicamente los datos de **USUARIOS**.

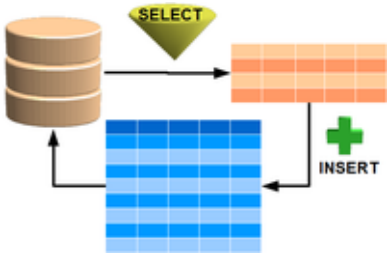
Una **subconsulta** puede obtener primero los usuarios que aparecen como creadores de partidas. Ese resultado podrá utilizarse después en la sentencia **UPDATE** para modificar únicamente los registros correspondientes.

De esta forma, una orden de edición puede basarse en información procedente de **otras tablas** sin tener que realizar manualmente varias operaciones independientes.

5.1.- Inserción de registros a partir de una consulta.

La **inserción de registros a partir de una consulta** permite utilizar el resultado de una sentencia **SELECT** para añadir una o varias **filas** a otra tabla mediante **INSERT**.

La sentencia **INSERT** no solo permite introducir valores escritos directamente mediante **VALUES**. También puede utilizar el resultado obtenido por una consulta **SELECT** para insertar registros.



Elaboración propia. *Inserción con subconsulta.* (CC BY-NC-SA)

Ejemplo:

Supongamos que existe una tabla **USUARIOS_SIN_CREDITO** con una estructura compatible con **USUARIOS** y queremos copiar en ella todos los usuarios cuyo crédito sea cero:

```
INSERT INTO USUARIOS_SIN_CREDITOSELECT *FROM USUARIOSWHERE Credito = 0;INSERT INTO USUARIOS
'VBROMI', 'NATALIA', 'SANCHEZ GARCIA','C/Blanca','28003','Madrid','Madrid','Spain', 'natsar
```

En este caso no se utiliza **VALUES**, ya que los valores que se insertarán proceden directamente del resultado de la consulta **SELECT**. Oracle admite esta forma para insertar las filas obtenidas por una consulta en otra tabla.

Es recomendable indicar las columnas de destino cuando no se desean insertar todos los campos o cuando interesa dejar clara su correspondencia.

Ejemplo:

```
INSERT INTO USUARIOS_SIN_CREDITO(Login, Nombre, Apellidos, Correo)SELECT Login, Nombre, Ape
'VBROMI', 'NATALIA', 'SANCHEZ GARCIA','C/Blanca','28003','Madrid','Madrid','Spain', 'natsar
```

Las columnas obtenidas mediante **SELECT** deben ser compatibles con las columnas indicadas en **INSERT** y mantenerse en el orden correspondiente.

También podemos utilizar una consulta para insertar únicamente los registros que cumplan una condición determinada.

Ejemplo:

Para copiar en la tabla **USUARIAS** todos los registros correspondientes a mujeres:

```
INSERT INTO USUARIASSELECT *FROM USUARIOSWHERE UPPER(Sexo) = 'M';INSERT INTO USUARIOS_SIN_C
```

Cada fila devuelta por la consulta **SELECT** generará una nueva fila en la tabla de destino.



Autoevaluación

La tabla **CLIENTES** contiene la columna **Nombre_CLI** y la tabla **NUEVOS_CLIENTES** contiene **Nombre_NCLI**. Se quieren insertar en **CLIENTES** todos los nombres almacenados en **NUEVOS_CLIENTES**.

¿Qué sentencia realiza correctamente la operación?

- ○ a. insert into nuevos_clientes (nombre_ncli) select nombre_cli from clientes;
- ○ b. insert into clientes (nombre_cli) values (select nombre_ncli from nuevos_clientes);
- ○ c. insert into clientes (nombre_cli) select nombre_ncli from nuevos_clientes;

Incorrecta. La operación está realizada en sentido contrario, ya que insertaría datos en **NUEVOS_CLIENTES** en lugar de hacerlo en **CLIENTES**.

Incorrecta. Cuando los valores proceden del resultado de una consulta **SELECT**, no se utiliza **VALUES** de esta forma.

Correcta. La consulta **SELECT** obtiene los valores de **Nombre_NCLI** y **INSERT** los incorpora en **Nombre_CLI** de la tabla **CLIENTES**.

Solución

1. Incorrecto
2. Incorrecto
3. Opción correcta

5.2.- Modificación de registros a partir de una consulta.

La **modificación de registros a partir de una consulta** permite utilizar el resultado de una **subconsulta** para calcular los nuevos valores o determinar qué registros deben actualizarse mediante **UPDATE**.

La sentencia **UPDATE** puede combinarse con consultas **SELECT** para realizar modificaciones basadas en información obtenida de la propia tabla o de otras tablas.

Una subconsulta puede utilizarse principalmente para:

- **Obtener** el nuevo valor que se asignará a una columna.
- **Determinar** qué registros deben modificarse.



Elaboración propia. *Actualización con subconsulta.* (CC BY-NC-SA)

Ejemplo:

La siguiente sentencia asigna a determinados usuarios el **mayor crédito existente** en la tabla **USUARIOS**. Solo se actualizan aquellos que aparecen como creadores de una partida cuyo estado es 1:

```
UPDATE USUARIOSSET Credito = (SELECT MAX(Credito) FROM USUARIOS)WHERE Login IN (    SELECT
USUARIOS) WHERE Login IN (SELECT Cod_Creador FROM PARTIDAS WHERE Estado=1);
```

La primera subconsulta:

```
SELECT MAX(Credito) FROM USUARIOS
```

obtiene el valor que se asignará al campo **Credito**.

La segunda:

```
SELECT Cod_Creador
FROM PARTIDAS
WHERE Estado = 1
```

Determina qué usuarios deben ser modificados.

Autoevaluación

La aplicación quiere aumentar en **50 créditos** a los usuarios que hayan creado al menos una partida con **Estado = 1**. El resto de usuarios debe conservar su crédito actual.

¿Qué sentencia realiza correctamente la operación?

○ a.

```
UPDATE USUARIOS
SET Credito = Credito + 50
WHERE Login IN (
    SELECT Cod_Creador
    FROM PARTIDAS
    WHERE Estado = 1
);
```

○ b.

```
UPDATE USUARIOS
SET Credito = 50
WHERE Login = (
    SELECT Cod_Creador
    FROM PARTIDAS
    WHERE Estado = 1
);
```

○ c.

```
UPDATE PARTIDAS
SET Credito = Credito + 50
WHERE Cod_Creador IN (
    SELECT Login
    FROM USUARIOS
);
```

Correcta. La subconsulta obtiene los **creadores de las partidas activas** y la sentencia **UPDATE** aumenta en 50 el crédito actual únicamente de esos usuarios.

Incorrecta. Credito = 50 sustituiría el crédito existente por 50 en lugar de aumentarlo. Además, la subconsulta puede devolver varios creadores, por lo que no corresponde utilizar **=** en este caso.

Incorrecta. La tabla que debe modificarse es **USUARIOS**, ya que es donde se almacena el crédito. **PARTIDAS** se utiliza para determinar qué usuarios cumplen la condición.

Solución

1. Opción correcta
2. Incorrecto
3. Incorrecto

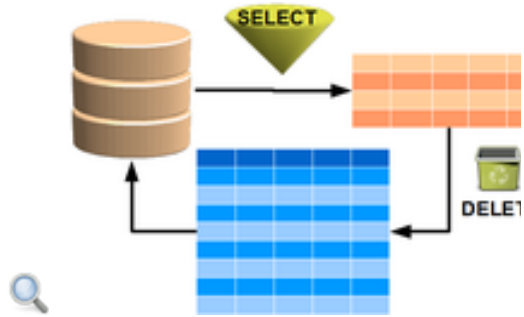
5.3.- Supresión de registros a partir de una consulta.

La **supresión de registros a partir de una consulta** permite utilizar el resultado de una **subconsulta** para determinar qué filas deben eliminarse mediante la sentencia **DELETE**.

La sentencia **DELETE** puede utilizar una consulta **SELECT** dentro de su condición para determinar qué registros deben eliminarse.

La estructura habitual es:

```
DELETE FROM nombre_tablaWHERE columna IN (    SELECT
```



Elaboración propia. *Borrado con subconsulta.* (CC BY-NC-SA)

La subconsulta se ejecuta para obtener los valores que se utilizarán posteriormente en la condición del **DELETE**. Oracle permite incluir subconsultas dentro de la cláusula **WHERE** de una sentencia **DELETE**.

Ejemplo:

Supongamos que queremos eliminar de **USUARIOS_SIN_CREDITO** aquellos usuarios que pertenecen a la provincia de Palencia. Podemos obtener primero sus identificadores mediante una consulta sobre **USUARIOS**:

```
DELETE FROM USUARIOS_SIN_CREDITOWHERE Login IN (    SELECT Login    FROM USUARIOS    WHERE
```

La subconsulta:

```
SELECT Login
FROM USUARIOS
WHERE Provincia = 'PALENCIA'
```

Obtiene los usuarios de Palencia, mientras que **DELETE** elimina de **USUARIOS_SIN_CREDITO** únicamente las filas cuyos valores de **Login** aparecen entre los resultados obtenidos.

Autoevaluación

La tabla **ANTIGUOS** contiene los códigos de usuarios que deben eliminarse de **USUARIOS**. Se quiere realizar el borrado mediante una **subconsulta**.

¿Qué elementos deben aparecer en la sentencia para realizar correctamente la operación?

- [ ] a. utilizar values para introducir los códigos que devuelve la consulta.

- [ ] b. utilizar delete from usuarios para indicar la tabla de la que se eliminarán los registros.

- [ ] c. utilizar delete from antiguos para borrar primero los códigos de la tabla auxiliar.

- [ ] d. utilizar una condición con in y una subconsulta que obtenga los códigos almacenados en antiguos.

Mostrar retroalimentación

Solución

1. Incorrecto
2. Correcto
3. Incorrecto
4. Correcto

6.- Transacciones.



Caso práctico



Mientras prueba la aplicación, **Ana** detecta un problema durante la creación de una partida.

—He iniciado una partida y la aplicación ha descontado el crédito del usuario —explica **Ana**—. Después he pulsado **Cancelar**, pero el crédito sigue reducido aunque la partida no se haya creado.

—Entonces esas operaciones no deberían tratarse de forma independiente —responde **Juan**—. El descuento del crédito y la creación de la partida forman parte de un mismo proceso. Si una de las operaciones no se completa, debemos poder **deshacer todas las anteriores**.

Para controlar este tipo de situaciones utilizarán **transacciones**.

Las **transacciones** agrupan una o varias **operaciones SQL** en una única unidad de trabajo, permitiendo **confirmar** todos sus cambios o **deshacerlos** si el proceso no puede completarse correctamente.

Una **transacción** es un conjunto de operaciones sobre la base de datos que se tratan como una **unidad de trabajo**. Su objetivo es evitar que un proceso quede realizado solo parcialmente y deje los datos en un estado incorrecto.

Las operaciones realizadas mediante sentencias **DML**, como **INSERT**, **UPDATE** o **DELETE**, pueden formar parte de una misma transacción.

Ejemplo:

Supongamos que se realiza una transferencia de 100 € entre dos cuentas.

El proceso requiere:

1. **Restar** 100 € del saldo de la cuenta de origen.
2. **Registrar** el movimiento de salida.
3. **Sumar** 100 € al saldo de la cuenta de destino.
4. **Registrar** el movimiento de entrada.

Si el sistema falla después de realizar únicamente las dos primeras operaciones, el dinero se habría descontado de una cuenta sin llegar a la otra.

*Una **transacción** permite considerar las cuatro operaciones como un único proceso:*

- **COMMIT.** Confirma los cambios realizados y los hace permanentes.
- **ROLLBACK.** Deshace los cambios pendientes de la transacción.

- **SAVEPOINT.** Establece un punto intermedio al que puede regresarse sin deshacer necesariamente toda la transacción.

De esta forma, si todas las operaciones finalizan correctamente, se realiza **COMMIT**. Si aparece un error antes de completar el proceso, puede utilizarse **ROLLBACK** para recuperar el estado anterior.

*Las transacciones se caracterizan por las propiedades conocidas como **ACID**:*

- **Atomicidad.** Todas las operaciones de la transacción se completan correctamente o no se aplica ninguna.
- **Consistencia.** La transacción debe llevar la base de datos de un estado válido a otro estado válido, respetando las reglas definidas.
- **Aislamiento.** Las transacciones concurrentes deben ejecutarse evitando que los cambios pendientes de una interfieran incorrectamente con las demás.
- **Durabilidad.** Una vez realizado **COMMIT**, los cambios confirmados deben conservarse de forma permanente.

Oracle también permite establecer determinadas características de una transacción mediante **SET TRANSACTION**, por ejemplo para trabajar en modo **READ ONLY**, y controlar cuándo se comprueban las restricciones aplazables mediante **SET CONSTRAINTS**.

> **Importante:** En Oracle, las sentencias **DDL**, como **CREATE**, **ALTER** o **DROP**, provocan confirmaciones implícitas, por lo que su ejecución afecta al control de la transacción.



Reflexiona

Durante la creación de una partida, la aplicación realiza estas operaciones:

1. **Reduce** en 50 el crédito del usuario.
2. **Inserta** la nueva partida.
3. **Registra** al usuario como participante.

Después de reducir el crédito, se produce un error y la partida no puede insertarse.

¿Por qué deberían formar las tres operaciones parte de una misma transacción y qué debería hacer la aplicación ante el error?

Mostrar retroalimentación

Las tres operaciones forman parte de un **mismo proceso lógico**. No tendría sentido reducir el crédito del usuario si finalmente la partida no llega a crearse.

Por ello, deberían ejecutarse dentro de una **transacción**. Si las tres operaciones terminan correctamente, los cambios pueden confirmarse mediante **COMMIT**. Si alguna falla antes de completar el proceso, debe utilizarse **ROLLBACK** para deshacer los cambios pendientes y recuperar el estado anterior.

De este modo se mantiene la **atomicidad** y se evita que la base de datos quede en un estado **inconsistente**.

6.1.- Hacer cambios permanentes.

Hacer cambios permanentes significa **confirmar una transacción** para que las modificaciones realizadas queden guardadas definitivamente en la base de datos mediante **COMMIT** o mediante una confirmación automática.

Los cambios realizados por sentencias **INSERT**, **UPDATE** o **DELETE** no son definitivos mientras la transacción permanezca pendiente. Para hacerlos permanentes se utiliza la sentencia **COMMIT**.

Ejemplo:

```
UPDATE USUARIOS
SET Credito = Credito + 50
WHERE Login = 'anamat56';

COMMIT;
```

Después de ejecutar **COMMIT**, los cambios quedan confirmados y ya no pueden deshacerse mediante **ROLLBACK**. Además, pasan a ser visibles para las demás transacciones que accedan posteriormente a esos datos.

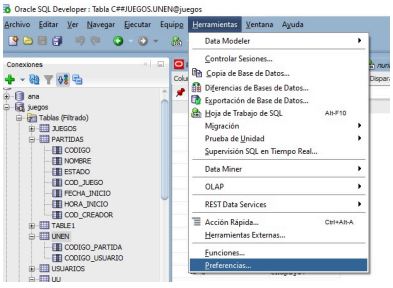
Oracle también realiza confirmaciones implícitas al ejecutar sentencias **DDL**, como **CREATE**, **ALTER** o **DROP**. Se realiza un **COMMIT** antes de la sentencia DDL y, si esta finaliza correctamente, otro después.

Confirmación automática

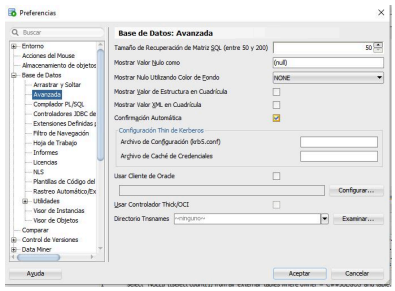
La opción **AUTOCOMMIT** permite confirmar automáticamente las operaciones sin tener que ejecutar manualmente **COMMIT** después de cada modificación.

Cuando **AUTOCOMMIT** está activado, cada operación DML realizada correctamente puede quedar confirmada automáticamente. En ese caso, ya no será posible utilizar posteriormente **ROLLBACK** para deshacerla.

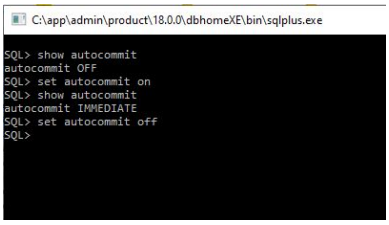
En **SQL Developer** puede configurarse la confirmación automática desde las preferencias de la base de datos.



Elaboración propia. *SQLDeveloper. Menu desplegado de opción Herramientas* (CC BY-NC-SA)



Elaboración propia. *Confirmación Automática SQLDeveloper.* (CC BY-NC-SA)



Elaboración propia. *Variable autocommit desde SQLPlus.* (CC BY-NC-SA)

En **SQL*Plus**, puede consultarse el estado de la confirmación automática mediante:

```
SHOW AUTOCOMMIT
```

Para activarla:

```
SET AUTOCOMMIT ON
```

Y para desactivarla:

```
SET AUTOCOMMIT OFF
```

Con **AUTOCOMMIT OFF**, los cambios deben confirmarse explícitamente mediante **COMMIT**. SQL*Plus mantiene actualmente estos comandos para controlar la confirmación automática.

> ***Importante:*** *En las aplicaciones es recomendable finalizar las transacciones de forma explícita mediante* ***COMMIT*** *o* ***ROLLBACK****, evitando depender del comportamiento que tenga la herramienta al cerrar la conexión.*



Autoevaluación

Indica si las siguientes situaciones son verdaderas o falsas.

1. Después de ejecutar COMMIT, los cambios confirmados pueden deshacerse mediante ROLLBACK mientras no se cierre SQL Developer.

○ Verdadero ○ Falso

Falso

COMMIT hace permanentes los cambios de la transacción, por lo que ya no pueden deshacerse posteriormente mediante **ROLLBACK**.

2. Con AUTOCOMMIT desactivado, una modificación realizada mediante UPDATE debe confirmarse explícitamente para que quede guardada de forma

permanente.

○ Verdadero ○ Falso

Verdadero

Con **AUTOCOMMIT OFF**, las operaciones DML permanecen pendientes hasta que se realiza **COMMIT** o se produce alguna situación que genere una confirmación implícita.

3. Ejecutar una sentencia DDL puede confirmar cambios DML pendientes realizados anteriormente en la misma sesión.

○ Verdadero ○ Falso

Falso

Oracle realiza un **COMMIT implícito antes de ejecutar una sentencia DDL**, por lo que los cambios DML pendientes anteriores pueden quedar confirmados.

6.2.- Deshacer cambios.

Deshacer cambios consiste en utilizar **ROLLBACK** para cancelar las modificaciones pendientes de la **transacción actual** y recuperar los datos anteriores a esas operaciones.

La sentencia **ROLLBACK** permite deshacer los cambios realizados durante la **transacción actual** que todavía no hayan sido confirmados mediante **COMMIT**. Al deshacer toda la transacción, esta queda finalizada.



Sasa Stefanovic. *Deshacer.* (CC BY-NC-SA)

Ejemplo:

Un usuario modifica por error el crédito de varios registros:

```
UPDATE USUARIOS
SET Credito = 0
WHERE Provincia = 'MADRID';
```

Si los cambios todavía no se han confirmado, pueden deshacerse mediante:

```
ROLLBACK;
```

De esta forma, los datos recuperan los valores que tenían antes de comenzar las modificaciones pendientes.

Una vez ejecutado **COMMIT**, los cambios son permanentes y **ROLLBACK ya no puede deshacerlos**.

Oracle recomienda finalizar explícitamente las transacciones mediante **COMMIT** o **ROLLBACK**. Si una aplicación o sesión termina de forma anormal sin haber confirmado la transacción, Oracle deshace automáticamente los cambios que permanecían pendientes.

Importante: El fallo de una sentencia SQL no implica necesariamente que se deshaga toda la transacción. Oracle puede deshacer únicamente los cambios producidos por la sentencia que ha fallado, manteniendo pendientes las operaciones anteriores.

Para saber más

Ejemplo sobre el uso de Rollback

Ejemplo de Rollback

Autoevaluación

¿Se pueden deshacer los cambios con la sentencia ROLLBACK después de que se haya ejecutado COMMIT?

- ○ Verdadero.
- ○ Falso.

Incorrecta. Después de ejecutar **COMMIT**, los cambios quedan confirmados y ya no pueden deshacerse mediante **ROLLBACK**.

Correcta. ROLLBACK solo puede deshacer los cambios que todavía permanecen pendientes en la transacción actual.

Solución

1. Incorrecto
2. Opción correcta

6.3.- Deshacer cambios parcialmente.

Deshacer cambios parcialmente consiste en establecer un **SAVEPOINT** dentro de una transacción para poder regresar posteriormente a ese punto sin cancelar todas las operaciones realizadas.

Un **SAVEPOINT** es un marcador que se establece dentro de una transacción para identificar un punto concreto al que se puede regresar posteriormente.

Resulta especialmente útil cuando una transacción contiene varias operaciones y se quiere disponer de la posibilidad de **deshacer solo una parte de los cambios**, conservando las operaciones realizadas anteriormente.



Sasa Stefanovic (Dominio público)

Ejemplo:

Supongamos que una transacción realiza las siguientes operaciones:

1. **Actualiza** el crédito de un usuario.
2. **Establece** un punto de restauración.
3. **Inserta** una nueva partida.
4. **Registra** al usuario como participante.

Si se produce un problema después del punto de restauración, pueden deshacerse las operaciones posteriores sin perder la actualización realizada anteriormente.

Para establecer el punto se utiliza:

```
SAVEPOINT partida_creada;SAVEPOINT nombre_punto_restauración;
```

Para regresar al punto establecido se utiliza:

```
ROLLBACK TO SAVEPOINT partida_creada;
```

Esta operación **deshace los cambios realizados después del SAVEPOINT**, pero mantiene los anteriores y **no finaliza la transacción**. Después pueden realizarse nuevas operaciones y decidir posteriormente si se utiliza **COMMIT** o un **ROLLBACK** completo.

Importante: Un **ROLLBACK** sin indicar un punto deshace toda la transacción pendiente, mientras que **ROLLBACK TO SAVEPOINT** permite deshacer solamente los cambios posteriores al punto seleccionado.

Para saber más

Ejemplo sobre el uso de los puntos de restauración.

Ejemplo de punto de restauración.

Actividad desplegable

Durante una transacción se realizan las siguientes operaciones:

1. Se reduce el crédito de un usuario.
2. Se establece un **SAVEPOINT** llamado **partida**.
3. Se crea una nueva partida.
4. Se detecta un error y se quiere conservar únicamente la reducción de crédito.

Selecciona la instrucción adecuada para cada necesidad:

Necesidad	Respuesta
1. Establecer un punto después de modificar el crédito.	
2. Deshacer la creación de la partida conservando los cambios anteriores.	
3. Deshacer todos los cambios pendientes de la transacción.	

Comprobar | Mostrar retroalimentación

1. **SAVEPOINT partida.** Correcto. Establece un marcador dentro de la transacción sin confirmar los cambios realizados.
2. **ROLLBACK TO SAVEPOINT partida.** Correcto. Deshace únicamente las operaciones realizadas después del punto **partida** y mantiene activa la transacción.
3. **ROLLBACK.** Correcto. Deshace todos los cambios pendientes de la transacción y la finaliza.

7.- Problemas asociados al acceso simultáneo a los datos. Concurrencia.

Caso práctico



Ana se plantea qué puede ocurrir cuando varios usuarios trabajan simultáneamente sobre los mismos datos.

—Si dos usuarios transfieren crédito al mismo tiempo a una tercera persona, ¿podría perderse una de las modificaciones? —pregunta **Ana**.

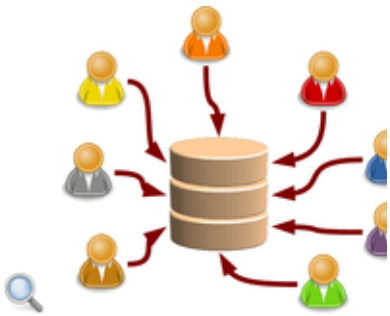
—Sí, si la aplicación no gestiona correctamente el acceso concurrente —responde **Juan**—. Por ejemplo, ambos procesos podrían leer inicialmente el mismo crédito y después intentar actualizarlo utilizando valores calculados a partir de esa información.

Si el usuario A tiene **0 créditos**, B quiere añadirle 100 y C quiere añadirle 50, ambos procesos podrían leer inicialmente el valor 0. Una implementación incorrecta podría terminar guardando 100 o 50 en lugar de los **150 créditos** que deberían resultar.

—Por eso necesitamos mecanismos de **control de concurrencia** —explica **Juan**—. La base de datos utiliza transacciones, versiones consistentes de los datos y bloqueos para coordinar los accesos simultáneos.

Los **problemas asociados al acceso simultáneo a los datos** aparecen cuando varias **transacciones** trabajan al mismo tiempo sobre información compartida. El **control de concurrencia** permite coordinar estos accesos para mantener la **consistencia** y la **integridad** de los datos.

En una base de datos **multiusuario**, diferentes sesiones pueden consultar o modificar información de forma simultánea. Esta capacidad recibe el nombre de **concurrencia**.



Sasa Stefanovic (Uso educativo nc)

El objetivo no es impedir que varios usuarios trabajen al mismo tiempo, sino conseguir que las transacciones concurrentes produzcan resultados **correctos y consistentes**.

Una base de datos multiusuario debe garantizar principalmente:

- **Concurrencia de datos**: Permite que varios usuarios accedan a la base de datos al mismo tiempo.
- **Consistencia de datos.** Cada usuario obtiene una visión coherente de la información, teniendo en cuenta sus propios cambios visibles y los cambios confirmados por otras transacciones.
- **Integridad de los datos.** Las operaciones concurrentes no deben provocar modificaciones incorrectas ni dejar la base de datos en un estado inválido.

Oracle combina diferentes mecanismos para controlar el acceso concurrente:

- **Consistencia de lectura.** Una consulta obtiene datos confirmados y coherentes correspondientes a un determinado momento. Oracle no permite leer cambios no confirmados de otra transacción.
- **Bloqueos.** Evitan que dos transacciones realicen determinadas modificaciones incompatibles sobre el mismo recurso al mismo tiempo.
- **Transacciones.** Agrupan las operaciones y permiten controlar cuándo los cambios se confirman o se deshacen.
- **Niveles de aislamiento.** Determinan cómo puede interactuar una transacción con los cambios realizados por otras.

En Oracle, los **bloqueos se gestionan normalmente de forma automática**. Cuando una transacción modifica una fila, esa fila queda bloqueada frente a otras modificaciones incompatibles hasta que finaliza la transacción. Sin embargo, otra sesión puede seguir consultando una **versión consistente** de los datos



Reflexiona

En la aplicación, dos usuarios intentan aumentar casi al mismo tiempo el crédito de **Ana**, que inicialmente tiene 100 créditos.

Una operación quiere añadir **40 créditos** y otra quiere añadir **60**. Ambas leen inicialmente el valor 100 y posteriormente intentan guardar el resultado calculado.

1. ¿Qué problema podría producirse si las dos operaciones no se coordinan correctamente?

2. ¿Cuál debería ser el crédito final de Ana?

3. ¿Por qué es necesario controlar la concurrencia aunque ambas operaciones sean válidas por separado?

Mostrar retroalimentación

Si ambos procesos calculan el nuevo crédito utilizando el mismo valor inicial, una actualización podría **sobrescribir** el resultado de la otra y provocar la pérdida de una modificación.

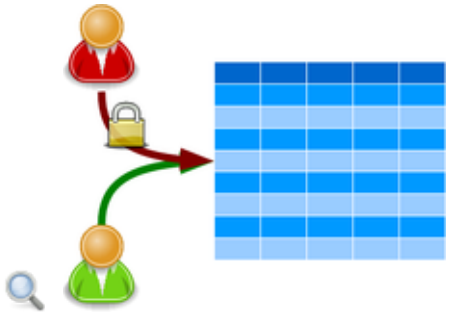
El resultado correcto debería ser **200 créditos**, ya que deben aplicarse ambos incrementos.

Aunque cada operación sea correcta individualmente, su ejecución simultánea puede producir un resultado incorrecto si no se coordina adecuadamente. El **control de concurrencia** permite que varias transacciones trabajen sobre datos compartidos manteniendo la **consistencia** y evitando interferencias entre sus modificaciones.

7.1.- Políticas de bloqueo.

Las **políticas de bloqueo** determinan cómo se coordinan los **lectores** y **escritores** cuando varias transacciones acceden simultáneamente a los mismos datos, evitando modificaciones incompatibles y manteniendo la **consistencia**.

Oracle gestiona automáticamente gran parte de los **bloqueos** necesarios para controlar el acceso concurrente a los datos. Un **lector** realiza una consulta, mientras que un **escritor** ejecuta una operación que modifica los datos.



Sasa Stefanovic. *Bloqueos compartidos y exclusivos* (CC BY-NC-SA)

Su comportamiento general puede resumirse de la siguiente forma:

- **Una fila se bloquea cuando se modifica.** Una operación como **UPDATE** obtiene un bloqueo sobre las filas afectadas.
- **Un escritor bloquea a otro escritor sobre la misma fila.** Si una transacción está modificando una fila, otra transacción que intente modificarla deberá esperar hasta que finalice la primera.
- **Un lector no bloquea a un escritor.** Una consulta normal puede leer una fila sin impedir que otra transacción la modifique.
- **Un escritor no bloquea a un lector.** Si una fila está siendo modificada, Oracle proporciona al lector una **versión consistente** anterior a los cambios que todavía no han sido confirmados.

Existe una excepción importante: **SELECT ... FOR UPDATE** permite seleccionar determinadas filas y **bloquearlas antes de modificarlas**, evitando que otra transacción pueda cambiarlas mientras se realiza el proceso.

Ejemplo:

Si una aplicación necesita consultar el crédito de un usuario y asegurarse de que ningún otro proceso lo modifica antes de actualizarlo, puede bloquear previamente esa fila mediante **SELECT ... FOR UPDATE**.

En el desarrollo de aplicaciones también pueden utilizarse dos estrategias habituales:

1. **Bloqueo pesimista.** Se bloquean los datos antes de realizar la modificación cuando existe una posibilidad importante de que otra transacción intente cambiarlos. **SELECT ... FOR UPDATE** es una forma de aplicar esta estrategia en Oracle.
2. **Control optimista.** Se permite trabajar sin mantener un bloqueo durante la lectura y, al actualizar, se comprueba que los datos no hayan sido modificados por otra transacción. Si han cambiado, la aplicación debe gestionar el conflicto.

El **control optimista** no es un tipo de bloqueo automático de Oracle, sino una estrategia que normalmente debe implementarse en la **lógica de la aplicación**.

Autoevaluación

Un usuario está modificando un registro y todavía no ha realizado **COMMIT**. En ese momento, otro usuario consulta ese mismo registro.

¿Qué información obtendrá el segundo usuario?

- ○ a. los cambios que está realizando el primer usuario aunque todavía no estén confirmados.
- ○ b. una versión consistente del registro sin los cambios pendientes del primer usuario.
- ○ c. ningún dato, porque la consulta quedará bloqueada hasta que termine la primera transacción.

Incorrecta. Oracle no permite que otra transacción consulte los **cambios no confirmados** realizados por un usuario.

Correcta. Oracle proporciona al lector una **versión consistente** del registro que no incluye los cambios pendientes de la otra transacción.

Incorrecta. Una modificación no bloquea normalmente una consulta. El lector puede acceder a una versión consistente de los datos mientras la otra transacción continúa activa.

Solución

1. Incorrecto
2. Opción correcta
3. Incorrecto

7.2.- Bloqueos compartidos y exclusivos.

Los **bloqueos compartidos y exclusivos** controlan qué operaciones pueden realizar simultáneamente distintas **transacciones** sobre un recurso, permitiendo determinados accesos concurrentes o reservando su modificación a una única transacción.

Oracle utiliza distintos tipos de **bloqueos** para controlar el acceso simultáneo a los datos.

- **Bloqueo exclusivo.** Impide que otras transacciones realicen modificaciones incompatibles sobre el recurso mientras se mantenga el bloqueo.



Sasa Stefanovic (Uso educativo nc)

Ejemplo:

```
LOCK TABLE nombreTabla IN EXCLUSIVE MODE;
```

Mientras se mantenga este bloqueo, otras transacciones podrán consultar la tabla, pero no realizar modificaciones incompatibles sobre ella.

- **Bloqueo compartido.** Permite que varias transacciones mantengan simultáneamente este tipo de bloqueo siempre que las operaciones sean compatibles.

Ejemplo:

```
LOCK TABLE PARTIDAS IN SHARE MODE;
```

Varias transacciones pueden mantener un bloqueo **SHARE** sobre la misma tabla al mismo tiempo, pero se impedirán operaciones incompatibles mientras permanezca activo.

Para saber más

Definición y ejemplos del bloqueo exclusivo y compartido

Integridad y control de concurrencia. (pdf - 307,08 KB) (0.30 MB)

Autoevaluación

Una aplicación está realizando una operación de mantenimiento sobre la tabla **PARTIDAS**. Durante ese proceso necesita impedir que otras transacciones puedan **modificar la tabla**, aunque sí puedan seguir realizando consultas normales.

¿Qué elementos encajan con esta necesidad?

- [ ] a. utilizar un bloqueo exclusive sobre la tabla partidas.
- [ ] b. utilizar lock table partidas in exclusive mode.
- [ ] c. impedir también cualquier consulta select mientras dure el bloqueo.
- [ ] d. mantener el bloqueo hasta que finalice la transacción.

Mostrar retroalimentación

Solución

1. Correcto
2. Correcto
3. Incorrecto
4. Correcto

7.3.- Bloqueos automáticos.

Los **bloqueos automáticos** son mecanismos que Oracle aplica sin intervención del usuario para proteger los **datos**, la **estructura de los objetos** y los **recursos internos** frente a operaciones concurrentes incompatibles.

Oracle obtiene automáticamente los **bloqueos** necesarios cuando una transacción realiza una operación que requiere proteger un determinado recurso. El tipo y nivel de bloqueo dependen de la operación realizada.

Los bloqueos automáticos se agrupan principalmente en:

- **Bloqueos DML.** Protegen los **datos** durante operaciones concurrentes. Sentencias como **INSERT**, **UPDATE** y **DELETE** bloquean las filas modificadas y obtienen también el bloqueo necesario sobre la tabla. Estos bloqueos se mantienen hasta realizar **COMMIT** o **ROLLBACK**.
  - **Ejemplo:** Si una transacción modifica una fila de **PARTIDAS** mediante **UPDATE**, Oracle bloquea automáticamente esa fila para impedir que otra transacción la modifique simultáneamente.
- **Bloqueos DDL.** Protegen la **definición de los objetos** mientras se realizan operaciones sobre su estructura. Se obtienen automáticamente cuando una sentencia DDL necesita proteger objetos como tablas o vistas.
  - **Ejemplo:** Durante una operación **ALTER TABLE**, Oracle utiliza los bloqueos necesarios para evitar operaciones incompatibles sobre la estructura de esa tabla.



Sasa Stefanovic. *Bloqueos automáticos.* (CC BY-NC-SA)

- **Bloqueos del sistema.** Protegen estructuras internas de la base de datos y de memoria. Se gestionan completamente de forma **automática** y el usuario no controla directamente su aparición ni su duración.
  - **Ejemplo:** Oracle puede utilizar bloqueos internos para coordinar el acceso a **estructuras de memoria** o a recursos necesarios para gestionar correctamente la ejecución simultánea de distintas operaciones.

Actividad desplegable

Una base de datos está realizando diferentes operaciones. Selecciona el tipo de bloqueo asociado a cada situación.

Situación	Tipo de bloqueo
1. Una transacción modifica una fila de **PARTIDAS** mediante UPDATE.	
2. Se modifica la estructura de una tabla mediante ALTER TABLE.	

Situación	Tipo de bloqueo
3. Oracle protege determinadas estructuras internas de memoria.	

Comprobar | Mostrar retroalimentación

1. **Bloqueo DML.** Las operaciones que modifican **datos** provocan automáticamente los bloqueos DML necesarios.
2. **Bloqueo DDL.** Las operaciones que actúan sobre la **estructura de los objetos** requieren bloqueos DDL.
3. **Bloqueo del sistema.** Estos bloqueos protegen los **recursos internos** de Oracle y son gestionados automáticamente por el sistema.

7.4.- Bloqueos manuales.

Los **bloqueos manuales** permiten solicitar de forma explícita la protección de determinadas **filas** o **tablas** cuando una transacción necesita controlar de manera específica el acceso concurrente a los datos.

Oracle gestiona los bloqueos de forma **automática** en las operaciones habituales. Sin embargo, determinadas aplicaciones pueden necesitar solicitar bloqueos explícitamente o establecer un nivel concreto de aislamiento.



Sasa Stefanovic (Uso educativo nc)

Entre las principales instrucciones utilizadas se encuentran:

- **LOCK TABLE.** Permite bloquear explícitamente una tabla utilizando el modo de bloqueo indicado.

Ejemplo:

```
LOCK TABLE PARTIDAS IN EXCLUSIVE MODE;
```

En este caso, otras transacciones podrán consultar **PARTIDAS**, pero no realizar modificaciones incompatibles mientras se mantenga el bloqueo.

- **SELECT ... FOR UPDATE.** Consulta determinadas filas y las bloquea para evitar que otra transacción las modifique antes de finalizar la operación.

Ejemplo:

```
SELECT Credito
FROM USUARIOS
WHERE Login = 'anamat56'
FOR UPDATE;
```

Las filas seleccionadas permanecen bloqueadas para otras modificaciones incompatibles hasta que la transacción realice **COMMIT** o **ROLLBACK**.

- **SET TRANSACTION ISOLATION LEVEL.** Permite establecer el **nivel de aislamiento** de la transacción y controlar cómo interactúa con otras transacciones concurrentes.

Ejemplo:

```
SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;
```

El nivel de aislamiento también puede establecerse para una sesión mediante **ALTER SESSION**.

Los bloqueos obtenidos explícitamente se liberan cuando la transacción finaliza mediante **COMMIT** o **ROLLBACK**.

Actividad desplegable

Una aplicación necesita consultar el crédito de un usuario y evitar que otra transacción lo modifique antes de actualizarlo. Después, la operación se confirma.

Completa los huecos con las opciones adecuadas:

SELECT CreditoFROM USUARIOSWHERE Login = 'anamat56' [ ] ;

UPDATE USUARIOSSET Credito = Credito + 50WHERE Login = 'anamat56';

[ ] ;

Comprobar Mostrar retroalimentación

a. **for update.** Correcta para el primer hueco. Permite bloquear las filas seleccionadas para evitar que otra transacción las modifique mientras la operación está en curso.
b. **commit.** Correcta para el segundo hueco. Confirma los cambios realizados y finaliza la transacción.

7.5.- Interbloqueos.

> Un **interbloqueo** se produce cuando dos o más **transacciones** quedan esperando mutuamente por recursos que mantienen **bloqueados**, impidiendo que puedan continuar.

Un **interbloqueo** o *deadlock* puede aparecer cuando varias transacciones modifican distintos recursos y posteriormente intentan acceder a los que están bloqueados por las otras.

Ejemplo:

Dos sesiones modifican diferentes usuarios:

Sesión 1:

```
UPDATE USUARIOS
SET Credito = Credito + 10
WHERE Login = 'anamat56';
```

Sesión 2:

```
UPDATE USUARIOS
SET Credito = Credito + 20
WHERE Login = 'alecam89';
```

Cada sesión mantiene bloqueada la fila que ha modificado.

A continuación, la **sesión 1** intenta modificar el registro bloqueado por la sesión 2:

```
UPDATE USUARIOS
SET Credito = Credito + 10
WHERE Login = 'alecam89';
```

Mientras tanto, la **sesión 2** intenta modificar el registro bloqueado por la sesión 1:

```
UPDATE USUARIOS
SET Credito = Credito + 20
WHERE Login = 'anamat56';
```

En ese momento:

- **Sesión 1.** Espera a que la sesión 2 libere su bloqueo.
- **Sesión 2.** Espera a que la sesión 1 libere el suyo.

Ninguna puede continuar y se produce un **interbloqueo**.

Oracle **detecta automáticamente** el interbloqueo y deshace una de las sentencias implicadas para liberar uno de los bloqueos. La sesión afectada recibe el error:

```
ORA-00060: deadlock detected while waiting for resource
```

Oracle deshace la **sentencia implicada**, no necesariamente toda la transacción, por lo que la aplicación debe gestionar posteriormente la situación mediante **ROLLBACK** o reintentando la operación cuando corresponda.

Una forma importante de reducir los interbloqueos es hacer que las transacciones accedan a los recursos **siguiendo siempre el mismo orden**.



Autoevaluación

Dos transacciones están trabajando simultáneamente:

1. La primera mantiene bloqueado el registro **A** y espera poder modificar **B**.
2. La segunda mantiene bloqueado **B** y espera poder modificar **A**.

¿Qué ocurrirá en Oracle?

- ○ a. ambas transacciones modificarán los registros simultáneamente para evitar la espera.
- ○ b. las dos permanecerán esperando indefinidamente hasta que una sesión sea cerrada manualmente.
- ○ c. oracle detectará el interbloqueo, deshará una de las sentencias implicadas y devolverá un error.

Incorrecta. Oracle no permite modificaciones incompatibles simultáneas sobre registros que están bloqueados por otra transacción.

Incorrecta. Oracle dispone de **detección automática de interbloqueos**, por lo que no mantiene indefinidamente esta espera circular.

Correcta. Oracle detecta el **interbloqueo**, deshace una de las sentencias implicadas para liberar el bloqueo y devuelve el error **ORA-00060**.

Solución

1. Incorrecto
2. Incorrecto

3. Opción correcta

Licencia: licencia propietaria